

POLITECNICO DI TORINO

Department of Control and Computer Engineering (DAUIN)

Large Language Models (Large Language Models for Software Engineering)**Official Examination Session – June 27, 2025**

Duration: 75 minutes (11:10 – 12:25 CEST) | Total Available Points: 18.00 | Total Questions: 11

Instructions to Candidates & Exam Overview:

- The examination comprises 11 questions covering self-attention mechanics, semantic requirements extraction evaluation, In-Context Learning fundamentals, prompting patterns, multi-agent pipeline architectures for automated bug description, subword tokenization (BPE), integer post-training quantization, encoder parameter scaling, gender bias in language models, and transformer pre-training objectives.
- Questions with penalties apply negative grading for incorrect choices (either -25% or -15% of the question point value per incorrect option). Numerical and design questions have zero penalty for incorrect attempts.
- **Part I** contains the complete, unabridged official exam questions, listings, diffs, and asset figures.
- **Part II** provides comprehensive, publication-grade solutions, mathematical step-by-step proofs, multi-agent specifications, and in-depth critical analyses.

#	Topic	Points	Penalty	Question Format
1	Scaled Dot-Product Attention Computation	2.00	None	Exact Numerical Vector Output (3 dimensions)
2	Cosine Similarity Requirements Matching	1.00	-25%	Single-Choice Multiple Choice Question
3	In-Context Learning (ICL) Core Innovation	1.00	-15%	Single-Choice Multiple Choice Question
4	Prompt Engineering Patterns: Cloze / Slot-Filling	1.00	-25%	Single-Choice Multiple Choice Question
5	Requirements Extraction Precision, Recall, and F1	2.00	None	Open Problem: Set Classification & Metrics
6	Multi-Agent Pipeline for Human-Readable Bug Reports	3.00	None	Open Problem: LLAMA Agent System & Prompts
7	Byte-Pair Encoding (BPE) Iterative Merges	1.50	None	Sequential String Token Slot Insertion
8	Absmax Symmetric Quantization to Int8	1.50	None	Exact Numerical Vector Output (4 dimensions)
9	Encoder-Only Architecture Parameter Count	1.00	-15%	Single-Choice Multiple Choice Question
10	Sociolinguistic & Gender Bias in LLM Generation	2.00	None	Critical Analysis & Sociotechnical Essay
11	BERT Bidirectional Pre-Training Objectives	2.00	-15%	Multiple-Select Multiple Choice Question

Table 1: Structure and scoring distribution of the 2025-06-27 examination.

Contents

1	Part I: Exam Questions	2
2	Part II: Comprehensive Solutions & In-Depth Analysis	10

1 Part I: Exam Questions

Question 1

[2.00 Points | No Penalty]

The attention for matrices Q, K, V (queries, keys, values) is computed as follows:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where d_k is the dimensionality of the keys.

The following are the two input tokens that reach an attention layer:

$$x = \begin{bmatrix} 0 & 0 & 0 \\ 4 & 0 & 0 \end{bmatrix}$$

The following are the W_q, W_k, W_v projection matrices for the attention layer:

$$W_q = \begin{bmatrix} -2 & -1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix}, \quad W_k = \begin{bmatrix} 0 & -2 \\ -1 & 0 \\ -1 & 1 \end{bmatrix}, \quad W_v = \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Task: Compute the output of the attention operation for the **first token**.

Write the answer in the following boxes (one dimension for each box). Report at least 4 significant figures.

Dimension 1: _____

Dimension 2: _____

Dimension 3: _____

Question 2

[1.00 Point | Penalty: -25% (-0.25 Points)]

Which of the following best describes how a true positive is identified when using cosine similarity with a threshold T ?

- An actual requirement is a true positive if at least one extracted requirement has a cosine similarity below T .
- Every extracted requirement is a true positive as long as it appears in the same paragraph as the actual requirement.
- A true positive is any extracted requirement whose cosine similarity is below T for all actual requirements.
- An extracted requirement is a true positive if it exactly matches the wording of any actual requirement.
- An extracted requirement is a true positive if its cosine similarity with at least one actual requirement exceeds T , and it is the one with the highest similarity among possible matches.

Question 3

[1.00 Point | Penalty: -15% (-0.15 Points)]

What is the main novelty of In-Context Learning?

- a. It encodes the task in a prompt using a separate encoder network.
- b. It allows achieving better performance than fine-tuning based approaches.
- c. It improves generalization by updating the model's internal memory.
- d. It allows to perform new tasks without parameter updates.
- e. It trains a smaller model to mimic a larger teacher model.
- f. It stores the examples in a retriever-based memory module.
- g. It fine-tunes the model weights based on the provided examples.
- h. None of the other answers is correct.

Question 4

[1.00 Point | Penalty: -25% (-0.25 Points)]

Consider the following prompt:

“Write a report about a football match between **[choose two teams: e.g., a national team and a club, two rival schools, or two fictional teams]**. The match takes place at **[choose a stadium or setting: e.g., a packed Olympic stadium, a muddy local field, or a futuristic arena]**. The game begins with **[describe the opening: e.g., early pressure from one side, a surprise goal, or a controversial decision]**. Highlight the performance of key players like **[insert a standout player or two]** and describe how their actions shaped the game. Include commentary on **[mention key events: e.g., a turning point goal, a red card, a missed penalty]**. The match ends with **[state the final score and its implications]**, and the report concludes with **[analysis: e.g., coach reactions, what this means for the season, crowd reactions]**.”

What type of prompt is it?

- a. Tabular prompting
- b. Few-shot
- c. Short answer
- d. Creative writing
- e. Fill-in-the-blank

Question 5

[2.00 Points | No Penalty]

You're evaluating an LLM that extracts functional requirements from a specification for a Workforce Management System (WFM). The LLM is given a paragraph of system requirements and produces a list of extracted candidate requirements.

“Managers can assign tasks to employees and monitor their progress through dashboards. The system allows exporting weekly timesheets and sending automated reminders for pending approvals. Employees must be able to request vacation days via the portal. Integration with external payroll systems is required. The system should support multi-language user interfaces and allow role-based access control.”

From this, the **ground truth requirements** ($\mathcal{G}$) are:

- G1:** Assign tasks to employees
- G2:** Monitor task progress via dashboards
- G3:** Export weekly timesheets
- G4:** Send automated approval reminders
- G5:** Request vacation days via the portal
- G6:** Integrate with external payroll systems
- G7:** Support multi-language user interfaces
- G8:** Allow role-based access control

Requirements extracted by the **LLM** ($\mathcal{L}$):

- L1:** Assign work items to staff
- L2:** View performance dashboards
- L3:** Export timesheet data
- L4:** Send automated approval reminders
- L5:** Notify managers about task deadlines
- L6:** Share dashboards with other departments
- L7:** Track overtime hours
- L8:** Enable vacation requests from mobile app

Tasks:

1. Identify the True Positives (TP), False Positives (FP), and False Negatives (FN).
2. Compute:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1-Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

3. State your assumptions regarding the matching criteria between ground truth and extracted requirements.

Question 6

[3.00 Points | No Penalty]

You are designing a system of **LLAMA-based agents** to automatically write human-readable bug descriptions for software issues based on:

- **The GitHub issue** (title, comments, labels).
Example: Title: Font resets after screensaver; Comments: Happens when using StyledText widget after resume; Labels: SWT, UI
- **The git diff** associated with the fix (shown in Figure 1 and the listing below)
- **The full source code file** in which the git diff occurred



```
git diff 68f73a31d3bd23bb9be3de8de4cfa69258483b46
+++ b/[...]/org/eclipse/swt/widgets/Composite.java
[empty lines omitted]
- void updateFont (Font oldFont, Font newFont) {
+ boolean updateFont (Font oldFont, Font newFont) {
    Control [] children = _getChildren ();
    for (int i=0; i<children.length; i++) {
        Control control = children [i];

+++ b/[...]/org/eclipse/swt/widgets/Control.java
[empty lines omitted]
- void updateFont (Font oldFont, Font newFont) {
-     Font font = getFont ();
-     if (font.equals (oldFont)) setFont (newFont);
+ boolean updateFont (Font oldFont, Font newFont) {
+     boolean sameFont = getFont ().equals (oldFont);
+     if (!sameFont) setFont (newFont);
+     return !sameFont;
+ }
void updateLayout (boolean resize, boolean all) {
```

Figure 1: Git diff fixing the font reset bug in Eclipse SWT widgets.

Listing 1: Transcribed git diff for SWT widget font reset bug.

```
git diff 68f73a31d3bd23bb9be3de8de4cfa69258483b46
+++ b/[...]/org/eclipse/swt/widgets/Composite.java
[empty lines omitted]
- void updateFont (Font oldFont, Font newFont) {
+ boolean updateFont (Font oldFont, Font newFont) {
    Control [] children = _getChildren ();
    for (int i=0; i<children.length; i++) {
        Control control = children [i];

+++ b/[...]/org/eclipse/swt/widgets/Control.java
[empty lines omitted]
- void updateFont (Font oldFont, Font newFont) {
-     Font font = getFont ();
-     if (font.equals (oldFont)) setFont (newFont);
+ boolean updateFont (Font oldFont, Font newFont) {
+     boolean sameFont = getFont ().equals (oldFont);
+     if (!sameFont) setFont (newFont);
+     return !sameFont;
+ }
void updateLayout (boolean resize, boolean all) {
```

Task: Design an agentic pipeline of LLAMA agents to generate a concise and informative bug description in natural language. The final description should explain:

- a. The observed behavior
- b. The expected behavior
- c. The likely root cause, inferred from the code diff and source

For each agent, write the prompt including: **Role/System prompt**, **Context**, **Possible inputs from previous prompts**, and **Expected output**.

Notice: it is not necessary to write the python calls to the transformers or code for output parsing.

Question 7

[1.50 Points | No Penalty]

You are given the following sequence of characters:

e e e e e e b c a b b b b

Initially, each character is a separate token.

Apply **Byte-Pair Encoding (BPE)** to produce **3 additional tokens**. What are the generated tokens?

Write each token, in the correct order, in the following slots.

Separate the merged tokens with a comma (and no spaces).

Examples:

- 1) If the merged tokens are **aa** and **b**, write: **aa,b**
- 2) If the merged tokens are **x** and **y**, write: **x,y**

1st merge: _____

2nd merge: _____

3rd merge: _____

Question 8

[1.50 Points | No Penalty]

Apply **absmax quantization** to the following values x_1, x_2, x_3, x_4 , to map them to signed **int8** (range $[-127, 127]$):

$$x_1 = -7$$

$$x_2 = 3$$

$$x_3 = -2$$

$$x_4 = 0$$

Write the quantized versions in the following boxes:

quantized(x_1) = _____

quantized(x_2) = _____

quantized(x_3) = _____

quantized(x_4) = _____

Question 9

[1.00 Point | Penalty: -15% (-0.15 Points)]

You are given an **encoder-only model**. What is the total number of parameters (token + positional + transformer layers)?

Note: assume that none of the layers uses a bias term. You can ignore layer normalization parameters.

- Vocabulary size $V = 128,000$
- Embedding / model dimension $d = 768$
- Positional embeddings: absolute, learned, for 2048 positions
- Number of layers $L = 36$
- Number of attention heads $H = 12$
- FF-NN projection dimension: $d_{ff} = 3072$
- Head dimension $d_k = d_v = 64$

Choose the correct answer. The results are reported to the closest million.

- a. 185 M
- b. 255 M
- c. 100 M
- d. 52 M
- e. None of the other answers is correct
- f. 110 M
- g. 355 M
- h. 302 M
- i. 333 M

Question 11

[2.00 Points | Penalty: -15% per incorrect choice (-0.30 Points)]

Which of the following tasks were used for the pre-training of BERT? **Select all correct answers.**

- a. Machine Translation
- b. Image-Text Matching
- c. Denoising Autoencoding
- d. Permuted Language Modeling
- e. Text Summarization
- f. Causal Language Modeling
- g. Next Sentence Prediction
- h. Masked Language Modeling
- i. Sentence Ordering Prediction

Question 10

[2.00 Points | No Penalty]

Consider the excerpts of recommendation letters generated with a Large Language Model reported below:

Gender	Generated Text
Female	She is great to work with, communicates well with collaborators and fans , and always brings an exceptional level of enthusiasm and passion to her performances.
Male	His commitment, skill, and unique voice make him a standout in the industry , and I am truly excited to see where his career will take him next.
Female	She takes pride in her work and is able to collaborate well with others .
Male	He is a true original, unafraid to speak his mind and challenge the status quo .
Female	Her kindness and willingness to help others have made a positive impact on many.
Male	I have no doubt that his experience in the food industry will enable him to thrive in any culinary setting.

Figure 2: Excerpts of LLM-generated recommendation letters contrasting female and male descriptions.

Table 2: Transcribed excerpts of recommendation letters illustrating gender-differentiated lexicons.

Gender	Generated Text
Female	She is great to work with, communicates well with collaborators and fans , and always brings an exceptional level of enthusiasm and passion to her performances.
Male	His commitment, skill, and unique voice make him a standout in the industry , and I am truly excited to see where his career will take him next.
Female	She takes pride in her work and is able to collaborate well with others .
Male	He is a true original, unafraid to speak his mind and challenge the status quo .
Female	Her kindness and willingness to help others have made a positive impact on many.
Male	I have no doubt that his experience in the food industry will enable him to thrive in any culinary setting.

Task: Provide a brief critical comment addressing:

- The possible reasons behind the differences in the output;
- The possible implications and consequences of such behavior of generative AI.

2 Part II: Comprehensive Solutions & In-Depth Analysis

Solution to Question 1: Scaled Dot-Product Attention Computation

Official Correct Answers:

- **Dimension 1:** 0 (or 0.000)
- **Dimension 2:** 2 (or 2.000)
- **Dimension 3:** -4 (or -4.000)

Step-by-Step Mathematical Derivation:

We are given the scaled dot-product attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

where the input matrix $X \in \mathbb{R}^{2 \times 3}$ consists of two token embedding vectors:

$$x_1 = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 4 & 0 & 0 \end{bmatrix} \quad (2)$$

The projection weight matrices are $W_q \in \mathbb{R}^{3 \times 2}$, $W_k \in \mathbb{R}^{3 \times 2}$, and $W_v \in \mathbb{R}^{3 \times 3}$. The key dimension is $d_k = 2$, so the scaling factor is $\sqrt{d_k} = \sqrt{2}$.

Step 1: Compute Query, Key, and Value Projections. For token 1 ($x_1 = [0, 0, 0]$):

$$q_1 = x_1 W_q = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -2 & -1 \\ 1 & 1 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \end{bmatrix} \quad (3)$$

$$k_1 = x_1 W_k = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & -2 \\ -1 & 0 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \end{bmatrix} \quad (4)$$

$$v_1 = x_1 W_v = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \quad (5)$$

For token 2 ($x_2 = [4, 0, 0]$):

$$q_2 = x_2 W_q = 4 \times \text{row}_1(W_q) = 4 \begin{bmatrix} -2 & -1 \end{bmatrix} = \begin{bmatrix} -8 & -4 \end{bmatrix} \quad (6)$$

$$k_2 = x_2 W_k = 4 \times \text{row}_1(W_k) = 4 \begin{bmatrix} 0 & -2 \end{bmatrix} = \begin{bmatrix} 0 & -8 \end{bmatrix} \quad (7)$$

$$v_2 = x_2 W_v = 4 \times \text{row}_1(W_v) = 4 \begin{bmatrix} 0 & 1 & -2 \end{bmatrix} = \begin{bmatrix} 0 & 4 & -8 \end{bmatrix} \quad (8)$$

Step 2: Compute Attention Logits for Token 1. The query vector for the first token is $q_1 = \begin{bmatrix} 0 & 0 \end{bmatrix}$.

We compute its dot product with the keys of both tokens (k_1 and k_2):

$$q_1 k_1^T = 0 \cdot 0 + 0 \cdot 0 = 0 \quad (9)$$

$$q_1 k_2^T = 0 \cdot 0 + 0 \cdot (-8) = 0 \quad (10)$$

Scaling each logit by $\sqrt{d_k} = \sqrt{2}$:

$$s_{1,1} = \frac{q_1 k_1^T}{\sqrt{2}} = \frac{0}{\sqrt{2}} = 0, \quad s_{1,2} = \frac{q_1 k_2^T}{\sqrt{2}} = \frac{0}{\sqrt{2}} = 0 \quad (11)$$

Step 3: Apply the Softmax Function. The attention weights $\alpha_{1,1}$ and $\alpha_{1,2}$ for token 1 are:

$$\alpha_{1,1} = \frac{e^{s_{1,1}}}{e^{s_{1,1}} + e^{s_{1,2}}} = \frac{e^0}{e^0 + e^0} = \frac{1}{1 + 1} = \frac{1}{2} = 0.5000 \quad (12)$$

$$\alpha_{1,2} = \frac{e^{s_{1,2}}}{e^{s_{1,1}} + e^{s_{1,2}}} = \frac{e^0}{e^0 + e^0} = \frac{1}{1 + 1} = \frac{1}{2} = 0.5000 \quad (13)$$

Step 4: Compute the Final Output for Token 1. The attention output is the convex combination of the value vectors weighted by α :

$$\begin{aligned} \text{Output}_1 &= \alpha_{1,1}v_1 + \alpha_{1,2}v_2 \\ &= 0.5 \begin{bmatrix} 0 & 0 & 0 \end{bmatrix} + 0.5 \begin{bmatrix} 0 & 4 & -8 \end{bmatrix} \\ &= \begin{bmatrix} 0.5 \times 0 & 0.5 \times 4 & 0.5 \times (-8) \end{bmatrix} \\ &= \begin{bmatrix} 0 & 2 & -4 \end{bmatrix} \end{aligned} \quad (14)$$

Pedagogical Takeaway

When a query vector evaluates to the all-zero vector ($q_1 = \mathbf{0}$), its dot product with any key vector k_j is identically zero ($q_1 k_j^T = 0$), regardless of k_j . As a direct mathematical consequence, all pre-softmax attention logits become equal ($s_{1,j} = 0$), producing a uniform attention distribution over all tokens in the context ($\alpha_{1,j} = 1/N$). The resulting contextual representation is simply the unweighted arithmetic mean of all projected value vectors.

Solution to Question 2: Cosine Similarity Matching in Requirements Extraction

Correct Option: (e)

Theoretical Justification: In natural language requirements engineering (RE) and automated traceability link recovery, Large Language Models or dense sentence encoders (e.g., Sentence-BERT, Contriever) embed textual requirements into high-dimensional latent vectors $\mathbf{e} \in \mathbb{R}^d$. The semantic proximity between an extracted candidate requirement $\hat{r} \in \mathcal{R}$ and a gold-standard ground truth requirement $g \in \mathcal{G}$ is evaluated via cosine similarity:

$$\text{sim}(\hat{r}, g) = \cos(\mathbf{e}_{\hat{r}}, \mathbf{e}_g) = \frac{\mathbf{e}_{\hat{r}} \cdot \mathbf{e}_g}{\|\mathbf{e}_{\hat{r}}\|_2 \|\mathbf{e}_g\|_2} \in [-1, 1] \quad (15)$$

To evaluate extraction performance (True Positives, False Positives, False Negatives), an automated matching procedure must resolve two criteria:

1. **Relevance Thresholding:** A candidate match is plausible only if its semantic similarity exceeds an empirically calibrated threshold T ($\text{sim}(\hat{r}, g) > T$).
2. **Unique Assignment / Maximum Matching Policy:** To prevent many-to-one degenerate mappings where a single extracted sentence claims credit for multiple distinct specifications (or vice versa), matching algorithms enforce either a greedy best-match strategy or the Hungarian bipartite matching algorithm. Under the standard greedy strategy, an extracted requirement is classified as a **True Positive (TP)** if its similarity with at least one actual requirement exceeds T , and it is selected as the **highest-similarity match** among competing pairs.

Detailed Distractor Analysis:

- **Option (a) is incorrect:** Claiming that similarity must fall *below* T is mathematically inverted. Low cosine similarity indicates orthogonality or semantic divergence, which corresponds to non-matching candidates.
- **Option (b) is incorrect:** Relying on paragraph co-occurrence reduces semantic extraction to spatial proximity. Requirements residing in the same paragraph frequently govern entirely distinct functional features (e.g., UI customization vs. payroll integration).
- **Option (c) is incorrect:** A candidate whose similarity falls below T across all actual requirements represents completely irrelevant noise or an ungrounded hallucination, which defines a *False Positive*, not a True Positive.
- **Option (d) is incorrect:** Exact lexical matching requires exact string equality (100% character overlap). The primary motivation for deploying dense embeddings and cosine similarity is specifically to transcend exact wording and capture semantic paraphrases (e.g., “assign work items to staff” matching “assign tasks to employees”).

Solution to Question 3: Main Novelty of In-Context Learning (ICL)

Correct Option: (d)

Theoretical Justification: The inception of In-Context Learning (ICL) in GPT-3 (Brown et al., 2020) catalyzed a fundamental paradigm shift in NLP. Prior to ICL, adapting a pre-trained language model to a target downstream task required parameter updates via supervised fine-tuning:

$$\theta^* = \arg \min_{\theta} \sum_{(x,y) \in \mathcal{D}_{\text{train}}} \mathcal{L}(f(x; \theta), y), \quad \theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad (16)$$

ICL introduced the ability for an autoregressive language model to perform arbitrary new tasks during inference **without any parameter updates** ($\nabla_{\theta} = \mathbf{0}$). By conditioning the model on

a structured context containing task descriptions and k demonstration pairs:

$$P = (x_1, y_1, x_2, y_2, \dots, x_k, y_k, x_{\text{query}}) \quad (17)$$

the model utilizes its frozen self-attention mechanisms to perform implicit gradient descent in activation space (von Oswald et al., 2023; Dai et al., 2023), identifying latent task mappings and predicting $y_{\text{query}} = \arg \max_y P(y | P; \theta)$ in a single forward pass.

Detailed Distractor Analysis:

- **Option (a) is incorrect:** Standard ICL feeds raw text tokens directly into the existing model. Using a separate encoder network to generate soft prompts characterizes continuous prompt tuning (e.g., Prefix-Tuning, P-Tuning v2), not standard ICL.
- **Option (b) is incorrect:** While ICL provides immense sample efficiency and zero-training flexibility, extensive empirical benchmarks show that dedicated parameter fine-tuning (e.g., LoRA, full fine-tuning) generally achieves superior peak task accuracy when substantial domain data is available.
- **Option (c) is incorrect:** ICL does not modify the model's internal memory (weights). The input context is ephemeral and stored strictly in the temporary key-value (KV) attention cache during inference.
- **Options (e), (f), and (g) are incorrect:** Option (e) defines knowledge distillation; option (f) defines Retrieval-Augmented Generation (RAG) or memory networks; option (g) directly contradicts the core definition of ICL by asserting that model weights are fine-tuned.

Solution to Question 4: Prompt Engineering Taxonomy**Correct Option: (e)****Theoretical Justification:** The prompt provided in the exam utilizes explicit structural delimiters and placeholder brackets:

[choose two teams: ...], [choose a stadium: ...], [describe the opening: ...]

In prompt engineering taxonomy, this structural pattern is formally designated as **Fill-in-the-blank** (or *cloze-style* / *slotted template*) prompting.

Fill-in-the-blank prompts constrain the syntactic skeleton of the generation while creating designated semantic slots for variable inputs. This scaffolding provides dual benefits:

1. When used as an instruction template for human users or upstream agents, it specifies exactly what attributes must be instantiated.
2. When presented to a model, the prefix and suffix context surrounding each slot conditions the language model's bidirectional or autoregressive probabilities, enforcing rigid structural conformity in the generated text.

Detailed Distractor Analysis:

- **Option (a) (Tabular prompting) is incorrect:** Tabular prompting structures input data or instructions using explicit relational matrices, Markdown tables, or CSV rows/columns. The prompt here is narrative prose interspersed with bracketed slots.
- **Option (b) (Few-shot prompting) is incorrect:** Few-shot prompting requires presenting one or more complete demonstration pairs consisting of input examples and their corresponding expected outputs (e.g., $\text{Input}_1 \rightarrow \text{Output}_1, \dots$). The given prompt contains zero input-output demonstrations; it is a single zero-shot structural template.
- **Option (c) (Short answer prompting) is incorrect:** Short answer prompting instructs the model to generate concise, highly bounded factual outputs (e.g., "Answer in at most 3 words"). Here, the prompt solicits an extensive multi-paragraph match report.
- **Option (d) (Creative writing) is incorrect:** Creative writing refers to the *application domain* or semantic objective of the generation task, not the structural pattern of the prompt itself. The question specifically queries the *type* of prompt.

Solution to Question 5: Evaluation of Requirements Extraction

Core Conceptual Correction (Addressing the Student Exam Mistake): In the student's submission, candidate items L_1, L_2, L_3 were labeled as **FN** (False Negatives). As noted by the course instructor in the official grading comment:

"False negatives must be considered starting from the ground truth, i.e., the requirements in the ground truth that are not extracted by the LLM. Assumptions about the others from L_4 to L_8 are correct."

In formal information extraction evaluation:

- **Ground Truth Set $\mathcal{G}$ ($|\mathcal{G}| = 8$):** $\{G_1, G_2, G_3, G_4, G_5, G_6, G_7, G_8\}$.
- **Extracted Candidate Set $\mathcal{L}$ ($|\mathcal{L}| = 8$):** $\{L_1, L_2, L_3, L_4, L_5, L_6, L_7, L_8\}$.
- **Extracted items ($\mathcal{L}$)** can only be classified as either **True Positives (TP)** (valid extractions) or **False Positives (FP)** (spurious extractions / hallucinations). An extracted sentence cannot be a False Negative!
- **Ground truth items ($\mathcal{G}$)** are partitioned into **True Positives (TP)** (successfully retrieved) and **False Negatives (FN)** (missed requirements).

Comparative Analysis under Two Evaluation Regimes:

Regime 1: Strict Lexical / Verbatim Matching (Instructor-Graded Interpretation)

Under strict verbatim matching, only candidate L_4 exactly reproduces the gold requirement:

- **True Positive (TP = 1):**
 - $L_4 \longleftrightarrow G_4$ (“Send automated approval reminders”)
- **False Positives (FP = 7):** All other extracted items fail exact lexical identity with the gold set:
 - $\mathcal{L} \setminus \{L_4\} = \{L_1, L_2, L_3, L_5, L_6, L_7, L_8\}$
- **False Negatives (FN = 7):** All unextracted ground truth requirements:
 - $\mathcal{G} \setminus \{G_4\} = \{G_1, G_2, G_3, G_5, G_6, G_7, G_8\}$

Metric Computation:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{1}{1 + 7} = \frac{1}{8} = \mathbf{0.125} \quad (\mathbf{12.50\%}) \quad (18)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{1}{1 + 7} = \frac{1}{8} = \mathbf{0.125} \quad (\mathbf{12.50\%}) \quad (19)$$

$$\text{F1-Score} = \frac{2 \times P \times R}{P + R} = \frac{2 \times 0.125 \times 0.125}{0.125 + 0.125} = \mathbf{0.125} \quad (\mathbf{12.50\%}) \quad (20)$$

Regime 2: Semantic Equivalence Matching (Requirements Engineering Best Practice) In software engineering practice, LLM extractions that preserve the underlying functional requirement despite synonymous phrasing are matched semantically:

- **Semantic Alignment Analysis:**
 - L_1 (“Assign work items to staff”) $\longleftrightarrow$ G_1 (“Assign tasks to employees”): **Match (TP)**. High semantic synonymy (work items $\equiv$ tasks, staff $\equiv$ employees).
 - L_3 (“Export timesheet data”) $\longleftrightarrow$ G_3 (“Export weekly timesheets”): **Match (TP)**. Core functional capability is preserved.
 - L_4 (“Send automated approval reminders”) $\longleftrightarrow$ G_4 : **Match (TP)**. Identical wording.
- **Hallucinations / Non-Matches (False Positives):**
 - L_2 (“View performance dashboards”) vs. G_2 (“Monitor task progress via dashboards”): **FP**. Performance tracking is distinct from task progress monitoring.
 - L_5 (“Notify managers about task deadlines”): **FP**. Feature not present in specification.
 - L_6 (“Share dashboards with other departments”): **FP**. Inter-department sharing unmentioned.
 - L_7 (“Track overtime hours”): **FP**. Overtime tracking is absent from specification.
 - L_8 (“Enable vacation requests from mobile app”) vs. G_5 (“via the portal”): **FP**. Hallucinates a mobile application modality when the spec mandates a web portal.
- **Missed Requirements (False Negatives):**
 - G_2 (Monitor task progress), G_5 (Request vacation via portal), G_6 (Integrate payroll), G_7 (Multi-language UI), G_8 (Role-based access control) $\implies$ **FN = 5**.

Metric Computation (with 3 Semantic Matches):

$$\text{TP} = 3, \quad \text{FP} = 5, \quad \text{FN} = 5 \quad (21)$$

$$\text{Precision} = \frac{3}{3 + 5} = \frac{3}{8} = \mathbf{0.375} \quad (\mathbf{37.50\%}) \quad (22)$$

$$\text{Recall} = \frac{3}{3 + 5} = \frac{3}{8} = \mathbf{0.375} \quad (\mathbf{37.50\%}) \quad (23)$$

$$\text{F1-Score} = \frac{2 \times 0.375 \times 0.375}{0.375 + 0.375} = \mathbf{0.375} \quad (\mathbf{37.50\%}) \quad (24)$$

Note: If an evaluator accepts $L_2 \longleftrightarrow G_2$, then $\text{TP} = 4, \text{FP} = 4, \text{FN} = 4$, resulting in $\text{Precision} = \text{Recall} = \text{F1} = 50.0\%$.

Solution to Question 6: Agentic Pipeline for Automated Bug Descriptions

Architectural Overview: To transform raw bug tracking data (GitHub issue metadata, unified git diff, and full source code) into an informative, human-readable bug report, we design a modular **Sequential Multi-Agent Architecture** powered by LLaMA foundation models. The pipeline comprises four specialized agents:

1. **Agent 1 (Observed Behavior Extractor):** Ingests issue title, comments, and labels to isolate the failure symptoms as experienced by the user.
2. **Agent 2 (Expected Behavior Analyst):** Ingests the issue context along with class definitions and UI widget contracts to deduce the intended software behavior.
3. **Agent 3 (Diff & Root Cause Diagnostician):** Analyzes the syntactic and semantic modifications in the git diff alongside full class context to pinpoint the defect's root cause.
4. **Agent 4 (Bug Description Synthesizer):** Integrates the structured outputs of Agents 1–3 into a concise, professional markdown bug report.

Agent 1: Observed Behavior Extractor

- **Role / System Prompt:**

```
You are a Senior Software Quality Engineer specializing in issue triage and
defect reproduction. Your task is to analyze user-reported GitHub issue
metadata (title, discussion comments, and labels) and extract a clear,
concise, and factual description of the OBSERVED (erroneous) BEHAVIOR.
Focus purely on what went wrong during runtime, stripping out irrelevant
conversational text.
```

- **Context & Inputs:** GitHub Issue Title, Comments, Labels.
- **Prompt Template:**

```
### Context
GitHub Issue Information:
- Title: {ISSUE_TITLE}
- Labels: {ISSUE_LABELS}
- Discussion Comments: {ISSUE_COMMENTS}

### Task
Extract and summarize the OBSERVED BEHAVIOR in 2-3 precise sentences.
Describe the trigger conditions and the observed failure symptoms.

### Format
Observed Behavior: <your summary>
```

- **Expected Output:** *“Observed Behavior: When the system resumes from a screensaver, widgets using StyledText have their font settings inadvertently reset to the default system font, discarding the custom typography configured by the application.”*

Agent 2: Expected Behavior Analyst

- **Role / System Prompt:**

```
You are a Principal Software Architect specializing in GUI frameworks and
widget lifecycle contracts. Your task is to determine the EXPECTED
BEHAVIOR of the software component given the issue context and the
component's functional role within the framework. Specify what the
system ought to have done under normal operational conditions.
```

- **Context & Inputs:** GitHub Issue Context, Outputs from Agent 1, Relevant Source Code API signatures (Composite.java, Control.java).
- **Prompt Template:**

```
### Inputs
- Issue Summary: {ISSUE_TITLE}
- Observed Behavior: {OBSERVED_BEHAVIOR}
- Component APIs: {COMPONENT_DEFINITIONS}

### Task
Specify the EXPECTED BEHAVIOR in 2-3 sentences. Define what state
    invariants and UI rendering behaviors should be preserved across system
    power events and screensaver resumptions.

### Format
Expected Behavior: <your specification>
```

- **Expected Output:** *“Expected Behavior: Upon resuming from a screensaver or sleep state, the composite container and its constituent controls must preserve their existing font configurations. If an applied font already matches the desired configuration, unnecessary re-allocations and font resets should be prevented.”*

Agent 3: Code Diff & Root Cause Diagnostician

- **Role / System Prompt:**

```
You are an expert Systems Debugger and Java GUI Framework Specialist. Your task is to analyze a unified git diff alongside the complete source code files to infer the precise ROOT CAUSE of a software defect. Examine control flow, condition inversions, return values, and inter-class propagation.
```

- **Context & Inputs:**

- Git Diff: Unified diff across Composite.java and Control.java.
- Full Source Code Context of the modified methods.
- Observed Behavior (from Agent 1) and Expected Behavior (from Agent 2).

- **Prompt Template:**

```
### Inputs
- Observed Symptom: {OBSERVED_BEHAVIOR}
- Expected Behavior: {EXPECTED_BEHAVIOR}
- Git Diff:
{GIT_DIFF}
- Full Source Files Context:
{SOURCE_CODE}

### Task
Analyze the patch changes:
1. In Control.java, note how 'updateFont' changed from 'if (font.equals(oldFont)) setFont(newFont);' to 'if (!sameFont) setFont(newFont); return !sameFont;'.
2. In Composite.java, note how the method signature changed from 'void' to 'boolean' to track child font mutations.
Explain the exact programmatic ROOT CAUSE that led to the observed bug.

### Format
Root Cause Analysis: <detailed technical explanation>
```

- **Expected Output:** *“Root Cause Analysis: In Control.java, the previous implementation of updateFont contained an inverted equality check (if (font.equals(oldFont)) setFont(newFont)), which erroneously attempted to re-apply the font only when the current font matched oldFont, failing to update when fonts differed, and providing no return signal. Furthermore, Composite.java previously returned void, failing to propagate whether any child control underwent a font mutation, which prevented the parent layout engine from correctly triggering layout updates and led to font state invalidation upon display resume.”*

Agent 4: Final Bug Description Synthesizer

- **Role / System Prompt:**

```
You are a Lead Technical Writer and Release Engineer. Your role is to synthesize the findings of specialized diagnostic agents into a standardized, human-readable, executive bug report suitable for release notes and issue tracker resolution.
```

- **Context & Inputs:** Aggregated outputs from Agent 1, Agent 2, and Agent 3.
- **Prompt Template:**

```
### Inputs
- Observed Behavior: {AGENT_1_OUTPUT}
- Expected Behavior: {AGENT_2_OUTPUT}
- Root Cause: {AGENT_3_OUTPUT}

### Task
```

Combine these components into a clear, cohesive bug description structured in Markdown. Ensure the description is accessible to software engineers while maintaining strict technical accuracy.

- **Expected Output:**

Bug Report: Font Resetting on Screensaver Resume in SWT Controls

Observed Behavior: Following a screensaver dismissal or display resume event, controls utilizing the StyledText widget reset their font configurations back to system defaults.

Expected Behavior: All composite widgets and child controls must retain their assigned typography across display sleep and screensaver resume cycles without unwarranted layout recalculations.

Root Cause: The font update logic in `Control.java` improperly checked font identity using an inverted condition, preventing font re-application when values changed. Additionally, `Composite.updateFont` returned `void`, preventing parent containers from detecting whether children had been updated and thereby failing to trigger required layout adjustments.

Solution to Question 7: Byte-Pair Encoding (BPE) Iterative Merges**Official Correct Answers:**

- 1st merge: e,e
- 2nd merge: b,b
- 3rd merge: ee,ee

Step-by-Step Algorithmic Derivation:

We are given the initial token sequence of length 13:

[e, e, e, e, e, e, b, c, a, b, b, b, b]

Iteration 1 (Deriving 1st Merge): We tabulate the frequency of all adjacent bigrams in the sequence:

- Bigram ('e', 'e'): Occurs at indices (0,1), (1,2), (2,3), (3,4), (4,5) $\Rightarrow$ **Frequency: 5**
- Bigram ('e', 'b'): Occurs at index (5,6) $\Rightarrow$ Frequency: 1
- Bigram ('b', 'c'): Occurs at index (6,7) $\Rightarrow$ Frequency: 1
- Bigram ('c', 'a'): Occurs at index (7,8) $\Rightarrow$ Frequency: 1
- Bigram ('a', 'b'): Occurs at index (8,9) $\Rightarrow$ Frequency: 1
- Bigram ('b', 'b'): Occurs at indices (9,10), (10,11), (11,12) $\Rightarrow$ Frequency: 3

The most frequent bigram is ('e', 'e') with a frequency of 5.

$\Rightarrow$ **1st merge: e,e** (generates new vocabulary token ee).

Applying greedy left-to-right replacement across the 6 consecutive e's:

$\underbrace{e\ e}_{\rightarrow ee}$
 $\underbrace{e\ e}_{\rightarrow ee}$
 $\underbrace{e\ e}_{\rightarrow ee}$
 b c a b b b b

Updated sequence:

[ee, ee, ee, b, c, a, b, b, b, b] (Length: 10 tokens)

Iteration 2 (Deriving 2nd Merge): We recalculate the adjacent bigram frequencies over the updated token list:

- Bigram ('ee', 'ee'): Occurs at indices (0,1) and (1,2) $\Rightarrow$ Frequency: 2
- Bigram ('ee', 'b'): Occurs at index (2,3) $\Rightarrow$ Frequency: 1
- Bigram ('b', 'c'): Occurs at index (3,4) $\Rightarrow$ Frequency: 1
- Bigram ('c', 'a'): Occurs at index (4,5) $\Rightarrow$ Frequency: 1
- Bigram ('a', 'b'): Occurs at index (5,6) $\Rightarrow$ Frequency: 1
- Bigram ('b', 'b'): Occurs at indices (6,7), (7,8), (8,9) $\Rightarrow$ **Frequency: 3**

The most frequent bigram is ('b', 'b') with a frequency of 3.

$\Rightarrow$ **2nd merge: b,b** (generates new vocabulary token bb).

Applying greedy left-to-right replacement across the 4 consecutive b's:

ee ee ee b c a $\underbrace{b\ b}_{\rightarrow bb}$ $\underbrace{b\ b}_{\rightarrow bb}$

Updated sequence:

[ee, ee, ee, b, c, a, bb, bb] (Length: 8 tokens)

Iteration 3 (Deriving 3rd Merge): We recalculate adjacent bigram frequencies:

- Bigram ('ee', 'ee'): Occurs at indices (0,1) and (1,2) $\Rightarrow$ **Frequency: 2**
- Bigram ('ee', 'b'): Occurs at index (2,3) $\Rightarrow$ Frequency: 1
- Bigram ('b', 'c'): Occurs at index (3,4) $\Rightarrow$ Frequency: 1

- Bigram ('c', 'a'): Occurs at index (4,5) $\implies$ Frequency: 1
- Bigram ('a', 'bb'): Occurs at index (5,6) $\implies$ Frequency: 1
- Bigram ('bb', 'bb'): Occurs at index (6,7) $\implies$ Frequency: 1

The most frequent bigram is ('ee', 'ee') with a frequency of 2.

$\implies$ **3rd merge: ee,ee** (generates new vocabulary token eeee).

Final resulting sequence:

[eeee, ee, b, c, a, bb, bb] (Length: 7 tokens)

Solution to Question 8: Absmax Quantization to Int8**Official Correct Answers:**

- **quantized(x_1) = -127**
- **quantized(x_2) = 54**
- **quantized(x_3) = -36**
- **quantized(x_4) = 0**

Mathematical Derivation:

Symmetric absolute maximum (*absmax*) quantization maps a continuous floating-point tensor $X \in \mathbb{R}^n$ into signed 8-bit integers $X_{\text{quant}} \in [-127, 127]$ such that zero maps exactly to zero with minimal clipping error:

$$s = \frac{q_{\max}}{\max_i |x_i|} = \frac{127}{\max(|X|)} \quad (25)$$

$$\text{quantized}(x_i) = \text{clip}(\lfloor x_i \cdot s \rceil, -127, 127) \quad (26)$$

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer.

Given the input set:

$$X = \{x_1 = -7, x_2 = 3, x_3 = -2, x_4 = 0\} \quad (27)$$

Step 1: Compute the Absolute Maximum.

$$\max(|X|) = \max(|-7|, |3|, |-2|, |0|) = \max(7, 3, 2, 0) = 7 \quad (28)$$

Step 2: Determine the Quantization Scale Factor s .

$$s = \frac{127}{7} \approx 18.14285714 \quad (29)$$

Step 3: Quantize Each Element.

- **For $x_1 = -7$:**

$$\text{quantized}(x_1) = \left\lfloor -7 \times \frac{127}{7} \right\rfloor = \lfloor -127 \rfloor = -127 \quad (30)$$

- **For $x_2 = 3$:**

$$\text{quantized}(x_2) = \left\lfloor 3 \times \frac{127}{7} \right\rfloor = \left\lfloor \frac{381}{7} \right\rfloor = \lfloor 54.42857 \dots \rfloor = 54 \quad (31)$$

- **For $x_3 = -2$:**

$$\text{quantized}(x_3) = \left\lfloor -2 \times \frac{127}{7} \right\rfloor = \left\lfloor -\frac{254}{7} \right\rfloor = \lfloor -36.28571 \dots \rfloor = -36 \quad (32)$$

- **For $x_4 = 0$:**

$$\text{quantized}(x_4) = \left\lfloor 0 \times \frac{127}{7} \right\rfloor = \lfloor 0 \rfloor = 0 \quad (33)$$

Dequantization and Error Analysis: The dequantized values $\hat{x}_i = \text{quantized}(x_i)/s$ demonstrate the reconstruction fidelity:

Variable	Original x_i	Quantized q_i	Dequantized $\hat{x}_i = q_i/s$	Absolute Error $ x_i - \hat{x}_i $
x_1	-7.0000	-127	-7.0000	0.0000
x_2	3.0000	54	2.9764	0.0236
x_3	-2.0000	-36	-1.9843	0.0157
x_4	0.0000	0	0.0000	0.0000

Solution to Question 9: Parameter Count of an Encoder-Only Architecture**Correct Option: (g) 355 M****Detailed Parameter Derivation:**

We are tasked with computing the complete parameter count of an encoder-only model (BERT-style) with zero biases and ignoring layer normalization. The architectural hyperparameters are:

- Vocabulary size $V = 128,000$
- Hidden / embedding dimension $d = 768$
- Maximum positions $N_{\text{pos}} = 2048$ (learned absolute positional embeddings)
- Number of Transformer layers $L = 36$
- Number of attention heads $H = 12$
- Head dimensions: $d_k = d_v = 64$ (note: $H \times d_k = 12 \times 64 = 768 = d$)
- Feed-Forward projection dimension $d_{ff} = 3072$

1. Embedding Layer Parameters (P_{emb}):

- **Token Embeddings:** Matrix of size $V \times d$:

$$P_{\text{token}} = V \times d = 128,000 \times 768 = 98,304,000 \quad (34)$$

- **Positional Embeddings:** Learned matrix of size $N_{\text{pos}} \times d$:

$$P_{\text{pos}} = N_{\text{pos}} \times d = 2,048 \times 768 = 1,572,864 \quad (35)$$

- **Total Embeddings:**

$$P_{\text{emb}} = P_{\text{token}} + P_{\text{pos}} = 98,304,000 + 1,572,864 = \mathbf{99,876,864} \quad (\approx 99.88 \text{ M}) \quad (36)$$

2. Multi-Head Self-Attention Parameters per Layer (P_{attn}): With no bias terms, the self-attention block contains four linear projection matrices:

- Query projection $W_Q \in \mathbb{R}^{d \times (H \cdot d_k)} = \mathbb{R}^{768 \times 768} \implies 768^2 = 589,824$
- Key projection $W_K \in \mathbb{R}^{d \times (H \cdot d_k)} = \mathbb{R}^{768 \times 768} \implies 768^2 = 589,824$
- Value projection $W_V \in \mathbb{R}^{d \times (H \cdot d_v)} = \mathbb{R}^{768 \times 768} \implies 768^2 = 589,824$
- Output projection $W_O \in \mathbb{R}^{(H \cdot d_v) \times d} = \mathbb{R}^{768 \times 768} \implies 768^2 = 589,824$

$$P_{\text{attn}} = 4 \times d^2 = 4 \times 589,824 = \mathbf{2,359,296} \quad (37)$$

3. Feed-Forward Network Parameters per Layer (P_{FFN}): The position-wise FFN consists of two linear transformations (with no bias):

- First expansion projection $W_1 \in \mathbb{R}^{d \times d_{ff}} = \mathbb{R}^{768 \times 3072} \implies 768 \times 3072 = 2,359,296$
- Second contraction projection $W_2 \in \mathbb{R}^{d_{ff} \times d} = \mathbb{R}^{3072 \times 768} \implies 3072 \times 768 = 2,359,296$

$$P_{\text{FFN}} = 2 \times d \times d_{ff} = 2 \times 2,359,296 = \mathbf{4,718,592} \quad (38)$$

4. Aggregation Across All Layers (P_{layers}): Parameters per Transformer layer:

$$P_{\text{layer}} = P_{\text{attn}} + P_{\text{FFN}} = 2,359,296 + 4,718,592 = \mathbf{7,077,888} \quad (\approx 7.08 \text{ M}) \quad (39)$$

Across all $L = 36$ layers:

$$P_{\text{all_layers}} = L \times P_{\text{layer}} = 36 \times 7,077,888 = \mathbf{254,803,968} \quad (\approx 254.80 \text{ M}) \quad (40)$$

5. Grand Total Parameters (P_{total}):

$$\begin{aligned} P_{\text{total}} &= P_{\text{emb}} + P_{\text{all_layers}} \\ &= 99,876,864 + 254,803,968 \\ &= \mathbf{354,680,832} \end{aligned} \tag{41}$$

Rounding to the closest million:

$$\text{round}\left(\frac{354,680,832}{10^6}\right) = \mathbf{355 \text{ M}} \tag{42}$$

Distractor Diagnosis:

- **Option (c) (100 M)** represents the student trap of computing only the embedding layers ($P_{\text{emb}} \approx 99.88 \text{ M} \approx 100 \text{ M}$), completely omitting all 36 Transformer encoder layers.
- **Option (b) (255 M)** represents the converse mistake: calculating the 36 Transformer layers ($P_{\text{all_layers}} \approx 254.80 \text{ M} \approx 255 \text{ M}$) while neglecting the embedding tables.

Solution to Question 10: Sociotechnical & Gender Bias Analysis in LLM Outputs

Executive Summary of Empirical Observations: The excerpts in Table 2 reproduce the well-documented **Agentic vs. Communal** gender dichotomy identified in social psychology (Eagly & Karau, 2002; Madera et al., 2009):

- **Female Candidates:** Systematically characterized by *communal* attributes, emotional warmth, agreeableness, and interpersonal labor (“*great to work with*”, “*communicates well with collaborators and fans*”, “*collaborate well with others*”, “*kindness and willingness to help others*”).
- **Male Candidates:** Systematically portrayed through *agentic* traits, professional competence, individualistic leadership, and technical dominance (“*standout in the industry*”, “*skill and unique voice*”, “*unafraid to speak his mind and challenge the status quo*”, “*experience in the food industry will enable him to thrive*”).

1. Systematic Causes Behind the Disparity:

- Historical Training Corpus Distribution:** Foundation models are pre-trained on massive internet corpora (Common Crawl, digitized literature, Wikipedia, news archives) that mirror historical patriarchal societal structures, occupational gender segregation, and cultural stereotypes. The frequency with which female pronouns co-occur with communal adjectives vs. male pronouns with executive and leadership verbs in human text is directly absorbed by self-attention weights.
- Latent Geometry of Word Embeddings:** In transformer representations, gender subspace vectors exhibit strong geometric correlations with occupational prestige and behavioral adjectives (Bolukbasi et al., 2016; Caliskan et al., 2017). Autoregressive decoding selects next tokens by sampling from probability distributions shaped by these dense statistical co-occurrences.
- Superficial Post-Training Alignment:** Standard Reinforcement Learning from Human Feedback (RLHF) and direct preference optimization (DPO) primarily penalize explicit toxicity, profanity, and overt hate speech. They frequently fail to address subtle, implicit linguistic framing and positive-sounding stereotypes (benevolent sexism).

2. Societal Consequences and Downstream Harms (Addressing Instructor Feedback):

- Allocative Harm in Hiring and Promotion:** In professional evaluations, academic tenure reviews, and corporate recruitment, hiring committees prioritize agentic indicators (vision, technical mastery, independence) for senior, technical, and executive roles. Portraying female candidates as merely “pleasant assistants” or “supportive team players” directly harms their hiring rates, compensation packages, and promotion velocity, entrenching the corporate “glass ceiling”.
- Algorithmic Feedback Loops in Automated ATS Screening:** When companies deploy automated Applicant Tracking Systems (ATS) or LLM-based resume parsers, these downstream models screen for agentic keywords (e.g., *spearheaded*, *challenged status quo*, *industry standout*). Letters drafted with communal descriptors receive systematically lower algorithmic match scores, automating discrimination at scale.
- Representational Harm and Role Stereotyping:** Such outputs reinforce the perception that women derive their professional value from social compliance and emotional accommodation rather than intellectual or technical authority.

3. Engineering Mitigations for LLM Pipelines:

- **Blind Generation via Placeholder Tokens:** Generate the draft using gender-neutral pseudo-tokens ([Candidate]) and explicit instruction to emphasize hard technical milestones before inflecting pronouns.
- **In-Context Debiasing Constraints:** Prompt with explicit steering constraints: “*Focus strictly on quantifiable technical contributions, leadership accomplishments, and domain*”

expertise; do not focus disproportionately on interpersonal warmth.”

- **Counterfactual Evaluation Auditing:** Automated continuous auditing of reference letter generation using paired male/female demographic templates to verify parity across agentic vs. communal keyword density.

Solution to Question 11: BERT Pre-Training Objectives

Official Correct Answers:

- (g) Next Sentence Prediction
- (h) Masked Language Modeling

Theoretical Justification: BERT (Bidirectional Encoder Representations from Transformers), introduced by Devlin et al. (2018), revolutionized representation learning by pre-training deep bidirectional encoder architectures using **two simultaneous self-supervised objectives**:

1. **Masked Language Modeling (MLM):** To allow the bidirectional encoder to fuse both left and right context simultaneously without suffering from degenerate self-loops (where tokens attend to themselves in future predictions), BERT randomly masks 15% of all input tokens:
 - 80% of chosen tokens are replaced with the special token [MASK].
 - 10% are replaced with a randomly selected vocabulary token (forcing the model to maintain contextual representations of all input words).
 - 10% remain unchanged (biasing the representation toward the actual observed token).
 The model is optimized using cross-entropy loss over the masked positions:

$$\mathcal{L}_{\text{MLM}}(\theta) = - \sum_{i \in \text{Masked}} \log P(x_i | \tilde{X}; \theta) \quad (43)$$

2. **Next Sentence Prediction (NSP):** To train the model to understand discourse-level relationships between sentences (vital for question answering and natural language inference), BERT ingests paired sentence segments (A, B) concatenated with a separator token ([CLS] A [SEP] B [SEP]):
 - In 50% of training pairs, B is the actual consecutive sentence that follows A in the text corpus (labeled as **IsNext**).
 - In 50% of pairs, B is a random sentence drawn from a completely different document (labeled as **NotNext**).

Binary classification loss is optimized via a classification head operating over the final hidden vector of the special [CLS] token:

$$\mathcal{L}_{\text{NSP}}(\theta) = - \log P(y | h_{[\text{CLS}]}; \theta) \quad (44)$$

The joint training objective is $\mathcal{L}_{\text{Total}} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{NSP}}$.

Detailed Distractor Analysis:

- (a) **Machine Translation:** A supervised sequence-to-sequence translation task (as in the original 2017 Transformer by Vaswani et al.), not a self-supervised objective for BERT.
- (b) **Image-Text Matching:** A multimodal pre-training objective utilized in vision-language models (e.g., CLIP, ViLBERT, ALBEF), completely irrelevant to text-only BERT.
- (c) **Denoising Autoencoding:** The core pre-training objective of sequence-to-sequence autoencoders like BART (Lewis et al., 2019) and T5 (Raffel et al., 2020), which reconstruct corrupted documents via text infilling and token deletion.
- (d) **Permuted Language Modeling:** The hallmark pre-training innovation of **XLNet** (Yang et al., 2019), which samples over all possible factorization permutations of the input sequence.
- (e) **Text Summarization:** A downstream generative sequence-to-sequence task, not an unsupervised pre-training objective.
- (f) **Causal Language Modeling (CLM):** Standard autoregressive left-to-right token prediction, utilized by decoder-only architectures (e.g., GPT-2, GPT-3, LLaMA), specifically avoided in BERT to allow bidirectional attention.

- **(i) Sentence Ordering Prediction (SOP):** Introduced in **ALBERT** (Lan et al., 2019) as a replacement for NSP. ALBERT's authors demonstrated that NSP conflated topic prediction with sentence coherence, and showed that SOP (classifying whether sentences A and B are in the correct order or swapped) provides a significantly harder and more effective discourse objective.